



Visitor

Lift an operation out of an object structure into a separate visitor, so you can add operations without editing the element classes.

 DIVE 01 · 10 km
Airplane porthole
The big picture & contours

→

 DIVE 02 · 1 km
Train window
Theses, examples, terms

→

 DIVE 03 · 100 m
Park walk
How the pieces relate

→

 DIVE 04 · 1 cm
Microscope
Exact wording & gotchas

 **Airplane porthole**

DIVE 01 · ORIENTATION

▼ 10 km

The double-dispatch specialist — it moves an operation out of the elements into a visitor, so new operations don’t touch the element classes.

— THE CONTOURS

• **Operations live in the visitor,**
not scattered across element classes

• **Double dispatch picks**
both the element type and the operation

• **Easy to add operations,**
painful to add new element types

— USE IT WHEN — GoF applicability

 **A stable structure of types**
needs many unrelated operations layered over it

 **Operations would otherwise pollute**
the element classes with unrelated concerns

Behavioral family: Strategy · Iterator · Command · Observer · Visitor — Iterator walks a structure, Visitor performs a type-specific operation on each node it reaches.

↓ DIVE DEEPER

 **Train window**

DIVE 02 · KEY OBJECTS

▼ 1 km

— THE THREE PARTS

Visitor
declares visitConcreteA() / visitConcreteB() — one method per element type

Element.accept(v)
the hook each element exposes; calls back v.visitX(this) — dispatch #2

ConcreteVisitor
implements an operation across all element types in one cohesive place

— IN THE WILD

 **TS compiler / Babel**
AST visitors run checks, transforms & code generation

 **ESLint rules**
each rule is a visitor keyed by AST node type

 **.NET Roslyn**
CSharpSyntaxVisitor / SyntaxWalker over the syntax tree

 **Apache POI / DOM**
document exporters visit nodes to emit HTML / PDF

```
<> element.accept(visitor) → visitor.visitConcrete(element) – the two calls form double dispatch.
```

GoF intent: “Represent an operation to be performed on the elements of an object structure.”

↓ DIVE DEEPER

 **Park walk**

DIVE 03 · CONNECTIONS

▼ 100 m

ObjectStructure
iterates its elements

→

element.accept(v)
dispatch on element type

→

v.visitConcrete(this)
dispatch on operation

→

✓ result
operation applied per node

— WHY IT FOLLOWS

→ **accept(v)** ⇒ the element resolves its own type and calls back the right visit method

→ **visitX(this)** ⇒ the visitor resolves the operation — together that’s double dispatch

→ **New visitor** ⇒ a whole new operation drops in without editing a single element class

🔗 Often pairs with Iterator (or Composite) to walk the structure · Interpreter trees are natural visitor targets · differs from Strategy by dispatching on element type, not just plugging one algorithm.

— STRUCTURE · UML

Visitor

```
# result: T

+ visitNumber(n: Num): void
+ visitBinary(b: Bin): void
+ visitConcrete(e: Element): void
```

Each Element.accept(v) calls back the matching visitX(this); the second call resolves the operation — that pair is double dispatch.

— WHAT IT BUYS YOU

✓ **Easy to add operations** — a new behavior = one new visitor class, elements untouched

✓ **Gathers related behavior** — one operation’s logic sits together in the visitor, not smeared across elements

✓ **Hard to add elements** — a new element type forces a new visit method in every visitor — the expression problem

✓ **Accumulates state** — the visitor can carry results across nodes — but that state is shared & order-sensitive

✓ **Breaks encapsulation** — visitors often need the elements’ internals, exposing more than the elements would like

↓ DIVE DEEPER

 **Microscope**

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Represent an operation to be performed on the elements of an object structure. Visitor lets you define a new operation without changing the classes of the elements on which it operates.”
– GoF, “Design Patterns”, 1994

— THE CODE

```
interface Visitor {
  visitNum(n: Num): number;
  visitAdd(a: Add): number;
}
class Num { constructor(public v: number) {}
accept(x: Visitor) { return x.visitNum(this); }
}
// element.accept → visitor.visitX(this):
// two dispatches, type + operation
const total = node.accept(new EvalVisitor());
```

— VARIANTS

Classic
double-dispatch visitor — accept() + one visit method per type

Pattern-matching
a switch over a tagged union — no accept(), TS/Scala/Rust style

Hierarchical
accept controls descent — visit enter/leave for nested structures

Acyclic
visitor has no fixed type list; accept casts — breaks the cyclic dep

Reflective
one visit(node) dispatches by runtime type via a map / instanceof

Default base
base visitor with no-op visits; subclasses override the few they need

— COMPARE THE ALTERNATIVES

Property	Visitor	Strategy	instanceof	Match
Add operation easily	✓	~	✓	✓
Add element type easily	✗	✓	~	~
Type-based dispatch	✓	✗	✓	✓
Compiler exhaustiveness	~	✗	✗	✓
Low boilerplate	✗	✓	✓	✓

Best fit → Visitor: stable types, many ops · Match: closed unions in TS/Rust · instanceof: a couple of types. (“~” = partial.)

— NUANCES & GOTCHAS

• **Double dispatch** — accept() resolves the element type, visitX() resolves the operation; together they pick the right method pair

• **Expression problem** — Visitor makes adding operations easy but adding an element type recompiles every visitor — the core trade-off

• **Encapsulation leak** — visitors usually need element internals, so accessors get exposed that the element wouldn’t otherwise publish

• **Result accumulation** — gather state in the visitor (a field or a return chain); mind that it’s shared and traversal-order dependent

• **Cyclic dependency** — Element knows Visitor and Visitor knows every Element; the two hierarchies are mutually coupled

• **vs Iterator** — Iterator only walks the structure; Visitor performs a type-specific operation at each node it reaches

— WATCH OUT

 **Anti-pattern:** Skip it when the element hierarchy is still churning — every new type breaks every visitor — or when there’s just one operation; a method on the element (or a simple switch) is simpler than the boilerplate.

 **Modern take:** In TS / Scala / Rust, pattern-matching over a discriminated union usually beats classic Visitor — same type-based dispatch, no accept() boilerplate, with exhaustiveness checks from the compiler.

— INTERVIEW PITFALLS

Q “What is double dispatch?”
Two virtual calls: element.accept(v) resolves the element’s type, then v.visitX(this) resolves the operation. The method that runs depends on both types at once.

Q “What’s the expression problem here?”
Visitor makes new operations cheap but new element types expensive — every visitor must gain a method. Pattern matching flips that trade-off.

Q “Why not just an if/instanceof chain?”
It works for a few types, but scatters each operation’s type checks and forgets to handle new types. Visitor centralizes an operation and (with a base) forces coverage.

Q “Main downsides?”
Boilerplate, a cyclic Element↔Visitor dependency, broken element encapsulation, and brittleness when element types change.

• Vasyi Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) **v1**